RoboLogic STEM Lab

System Architecture and Delivery Status

A senior full-stack architecture walkthrough of the current platform, what is implemented, how the pieces interact, why the boundaries exist, and what remains before Chemistry and Biology can be enabled.

Current release posture

Physics and RoboMaze are enabled. Creator Academy is RoboMaze-specific. Chemistry and Biology are registered as disabled extension points.

Executive summary

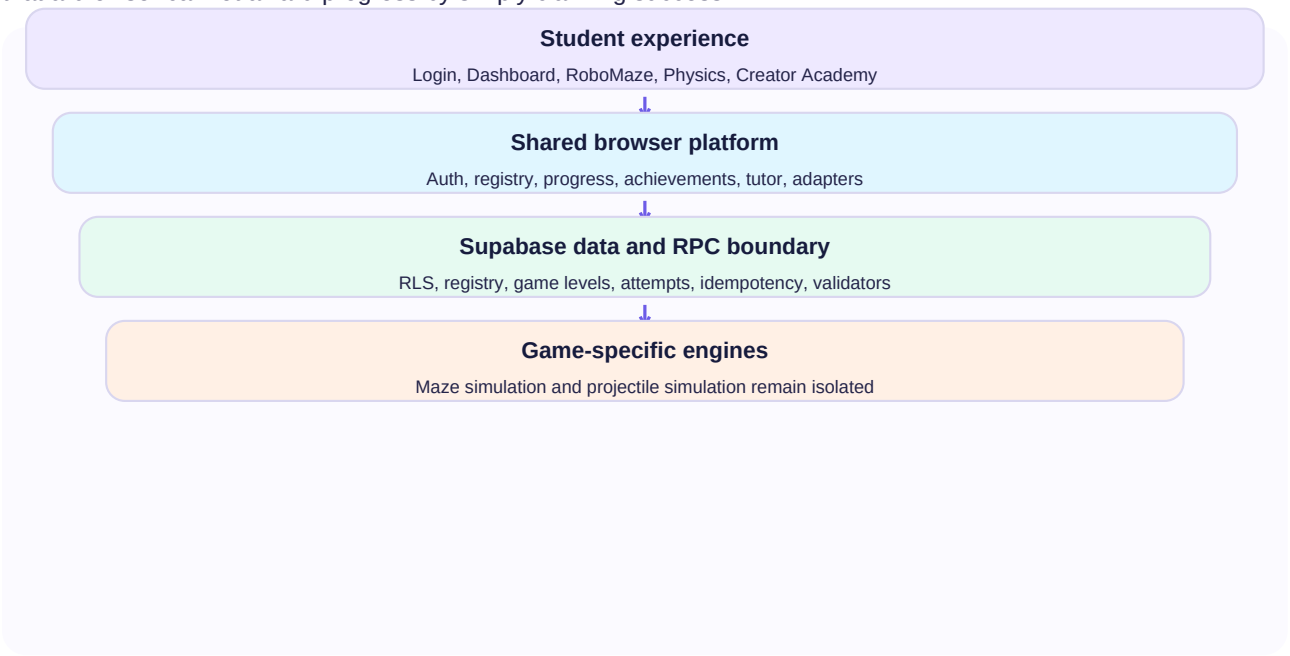
RoboLogic is a browser-first STEM learning platform with a shared learner shell and isolated game engines. Authentication, progress, achievements, tutor support, and safe attempt submission are centralized. Physics and RoboMaze own their simulations and validation rules. The new Phase 8 foundation adds a registry so future games can be added by metadata and module contracts rather than by duplicating shared platform logic.

Area	Current state
Enabled games	Physics and RoboMaze, each with a 15-level expo path and server-authoritative attempt validation.
Creator Academy	RoboMaze-only; visible levels are 16-20 while internal IDs remain 31-35 for compatibility.
Extension foundation	Game registry, feature flags, common attempt boundary, generic module contract, and registry-aware dashboard/progress logic.
Not yet enabled	Chemistry and Biology metadata exists, but real level content and authoritative validators are still required.

THE PLATFORM AT A GLANCE

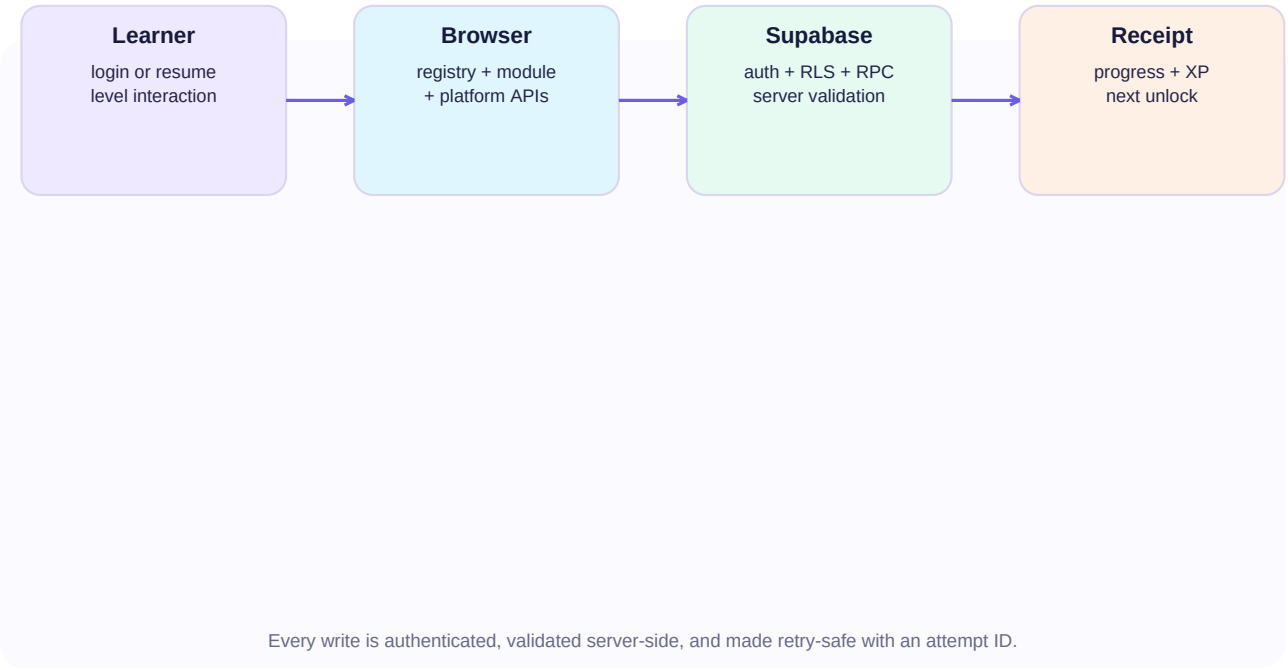
1. System architecture

The architecture is intentionally split into four layers. This keeps student-facing presentation flexible while ensuring that a browser cannot award progress by simply claiming success.



Layer	Responsibility	Why it exists
Student experience	Login, dashboard, game controls, visual feedback, Creator Academy.	Makes the learning loop understandable and rewarding for ages 11-14.
Shared browser platform	game-registry.js, game-platform.js, game-auth.js, progress-api.js, adapters, tutor and achievement APIs.	Prevents every game from reimplementing identity, progress, and error handling.
Supabase boundary	Auth, RLS, canonical game levels, RPCs, idempotency receipts, server-owned progression.	Makes completion, unlocks, XP, and stored results trustworthy.
Game engines	RoboMaze simulation/program runner and Physics projectile simulation.	Keeps domain rules testable and prevents cross-game coupling.

2. Runtime request and learning flow



The main path is deliberately one-directional: the UI gathers an attempt, the shared platform adds identity and a unique attempt ID, the server recomputes the result from canonical configuration, and only then does progress change.

Step	What happens	Owner
1. Authenticate	Supabase Auth establishes a session. Protected pages redirect unauthenticated learners to login.	game-auth.js + Supabase Auth
2. Load state	Progress API loads unlocked/completed levels and applies access to level pickers.	progress-api.js
3. Interact	The game engine runs locally for immediate animation and feedback.	RoboMaze or Physics module
4. Submit	The shared API validates the payload shape, creates a UUID attempt ID, and calls submit_game_attempt_idempotent.	progress-api.js
5. Validate	The common RPC dispatches to the existing Physics or RoboMaze authoritative validator.	Supabase RPC
6. Persist	Only server results update attempts, progress, achievements, XP, and next-level access.	Postgres + RLS

3. Backend and data architecture

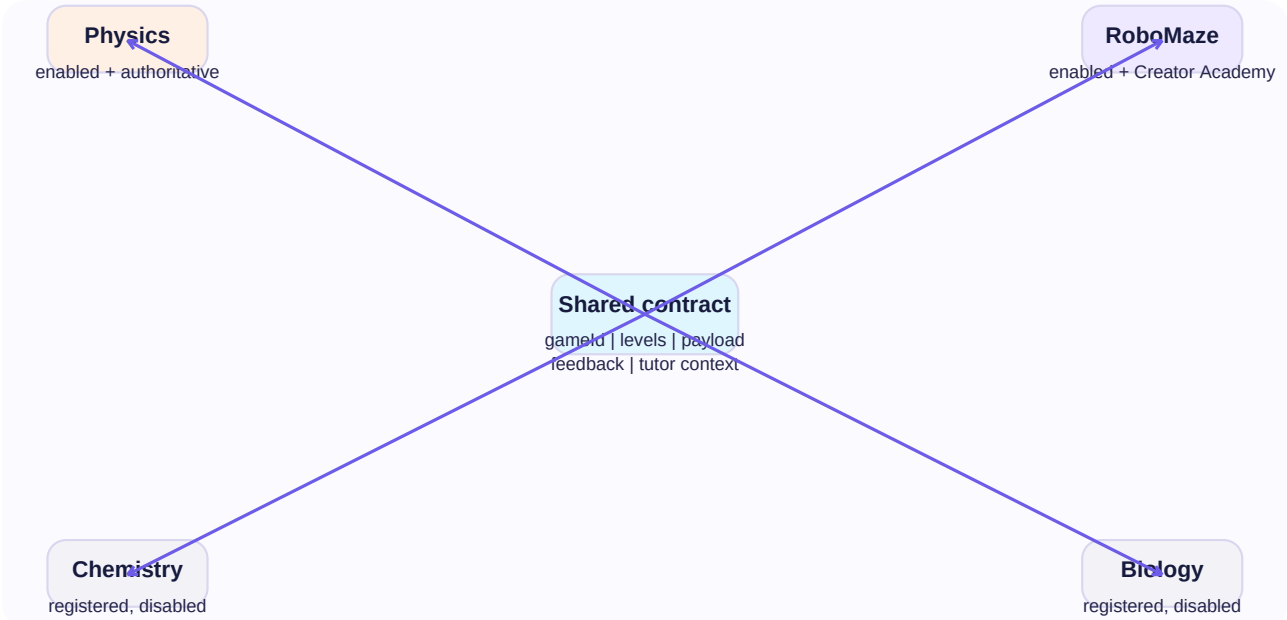
The database is the source of truth for canonical content and learner progression. The browser can request operations but cannot directly write progress, attempts, or achievements through ordinary table access.

Data object	Purpose	Ownership and protection
games	Registry metadata: stable ID, name, route, level count, validator key, icon, enabled flag.	Authenticated read; release flag prevents incomplete games from becoming playable.
game_levels	Canonical level title, instruction, and JSON configuration.	Seeded by migration; foreign keys keep progress tied to real levels.
level_progress	Locked/unlocked/completed state, best result, attempts.	Learner-owned reads; writes happen inside authoritative RPCs.
level_attempts	Every attempt and server result, including mistake code and validated details.	Learner-owned reads; server inserts only.
attempt_receipts	Idempotency record keyed by user, game, and attempt UUID.	No direct table access; RPC returns the original receipt on retries.
student_achievements	Best stars, XP, attempts, and completion result per level.	Server-owned achievement RPC; prevents client inflation.
tutor and mistakes	Mistake categorization, tutor context, quota state, and AI insight.	Session-bound requests and quota/refund hardening.

Migration sequence currently ends at 020. Migrations 019 and 020 add Chemistry/Biology IDs, registry metadata, and the common attempt boundary while preserving historical Physics/RoboMaze rows.

4. Game registry and module boundaries

The registry is the central decision point for routes, labels, level counts, validator availability, and release flags. It is intentionally data-driven, but it does not erase game-specific simulation logic.



Contract	Required behavior	Current status
Game registry	Stable gameId, route, display metadata, levelCount, enabled, validator.	Implemented for all four IDs. Chemistry/Biology disabled.
Game module	loadLevel, reset, getAttemptPayload, renderFeedback; optional mistake and tutor context builders.	Generic contract implemented; existing engines remain compatible.
Shared adapter	Unlock checks, checkpoint restore, attempt submit, mistake, tutor, achievement calls.	Existing adapter provides the shared boundary.
Validator	Recompute success from canonical level data; reject forged results; return stable receipt.	Physics and RoboMaze ready. Chemistry/Biology missing.

Why this design matters: a new game can be developed privately and remain invisible until its content, validator, adapter, and tests are all ready. Enabling a game becomes a controlled configuration change instead of a dashboard rewrite.

5. Existing game architecture

Game	Local engine	Server validation	Special features
RoboMaze	Grid simulation, command blocks, Python view, loop expansion, creator grid and program runner.	Canonical grid walk validator; server checks movement, walls, goal, command limits, and progression.	Creator Academy; visible Levels 16-20 map to internal IDs 31-35.
Physics	Canvas projectile simulation with angle, speed, drag, wall, trajectory, hint, and retry feedback.	Server recomputes trajectory from angle/speed/level configuration, including drag requirements and constraints.	15-level expo path; physics levels remain independent from RoboMaze.

The local simulation is for responsiveness and teaching feedback. It is not the authority for success. This distinction is the most important architectural rule in the system.

6. Security and trust model

The system treats every browser result as a hint, not a fact. A learner can control the UI and network request, so the server must recompute the outcome from canonical configuration.

Threat	Control	Result
Forged success	Authoritative validators ignore client success flags and simulate from payload plus canonical level data.	A forged request cannot unlock a level.
Replay/double submit	Client generates a UUID; receipts use user + game + attempt ID as a unique key.	Retries return the original result instead of awarding twice.
Unauthorized writes	RLS plus revoked direct RPC/table permissions; only authenticated server procedures can mutate progression.	A browser cannot directly insert progress or achievements.
Incomplete game exposure	Registry enabled flag and validator availability gate submissions and dashboard playability.	Chemistry/Biology remain disabled until safe.
Tutor abuse/cost	Quota reservation, explicit failure states, and refunds for provider failures.	Learners receive fallback feedback without uncontrolled API spend.

7. Delivery status and remaining work

Workstream	Status	Evidence / next action
Shared authentication	Done	Login, session checks, redirects, password visibility, and protected game pages.
Physics + RoboMaze	Done for current prototype	15-level paths, local simulations, authoritative RPC validators, progress and achievement flow.
Creator Academy	Done for RoboMaze	Five creator challenges, server validation, sharing, unlock gate, visible numbering 16-20.
Registry foundation	Done in code	Frontend registry plus migrations 019-020. Apply migrations to hosted Supabase.
Common submission boundary	Done in code	submit_game_attempt_idempotent dispatches to current validators. Hosted RPC verification remains.
Chemistry content	Not supplied	Need level definitions, UI module, payload shape, validator, and integration/browser tests.
Biology content	Not supplied	Need the same game-owned deliverables independently of Chemistry.
Hosted integration CI	Remaining	Use disposable Supabase project to apply migrations, seed levels, test RPC permissions, idempotency, and preservation.

Important distinction: the platform foundation is complete, but it would be misleading to call Chemistry and Biology complete without their actual educational content and authoritative validation rules.

8. Recommended rollout

1	**Apply and verify migrations 019-020**	Run against a disposable Supabase project first. Confirm existing progress and function permissions remain intact.
2	**Receive Chemistry and Biology contracts**	For each game: level JSON schema, attempt payload, success criteria, mistake categories, tutor context, and UI module.
3	**Implement validators first**	Write server validators and idempotent submission tests before enabling any student route.
4	**Add canonical levels**	Seed game_levels with the stable ID and validate parity against the module catalog.
5	**Integrate modules privately**	Use StemGameModule and StemGameAdapter; keep registry enabled=false while browser and hosted tests run.
6	**Enable independently**	Set one registry entry enabled at a time after hosted verification. Chemistry failure must not block Biology.

Architecture decision record

Keep shared platform responsibilities centralized, but keep simulation and validation rules owned by each game. Registry metadata controls visibility and routing; it does not become a generic substitute for domain logic. This gives the expo prototype stability today and a predictable onboarding path for future STEM games.